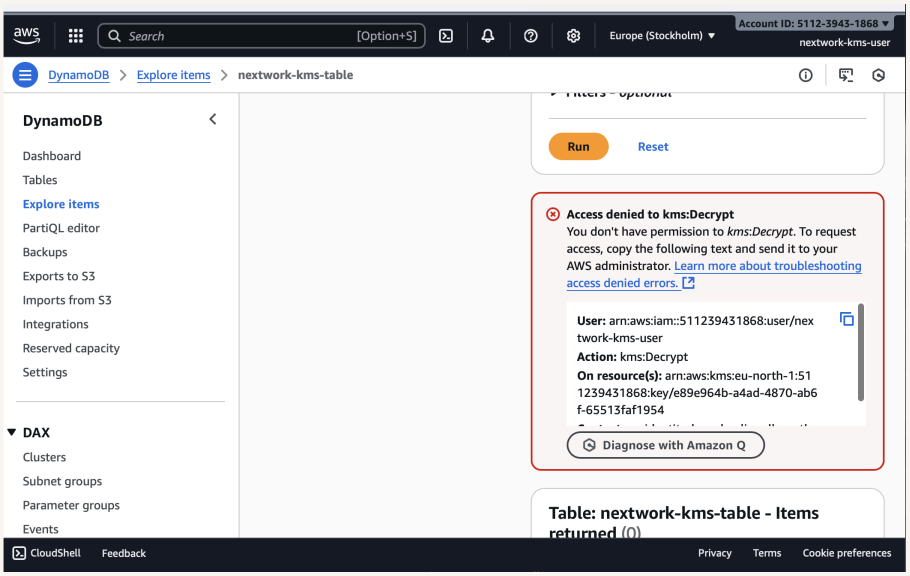
nextwork.org

Encrypt Data with AWS KMS

Kehinde Abiuwa



Kehinde Abiuwa
NextWork Student

Introducing Today's Project!

In this project, I will demonstrate creating a customer-managed KMS key, encrypting a DynamoDB table with it, writing/reading items to verify encryption, testing blocked access, and then granting a user the right encrypt/decrypt permissions. The goal is to show how KMS + DynamoDB protect data at rest and how key policies/IAM grants enforce least-privilege access end to end.

Tools and concepts

Services I used include AWS KMS (customer-managed key), Amazon DynamoDB& IAM. Key concepts I learnt include encryption at rest with SSE-KMS, envelope encryption (data keys), the difference between Key administrators vs Key users, least-privilege access with key policies/grants, request-time checks for kms:Decrypt/Encrypt/GenerateDataKey, transparent data encryption in DynamoDB, and auditing KMS calls to confirm access.

Project reflection

This project took me approximately 40 minutes. The most challenging part was getting the KMS key policy right—separating Key administrators from Key users and granting only kms:Decrypt/Encrypt/GenerateDataKey so DynamoDB would stop throwing AccessDenied. It was most rewarding to log in as the test user, read items successfully, and see matching KMS Decrypt events in CloudTrail—clear proof that encryption and least-privilege are working end to end.

Kehinde Abiuwa
NextWork Student

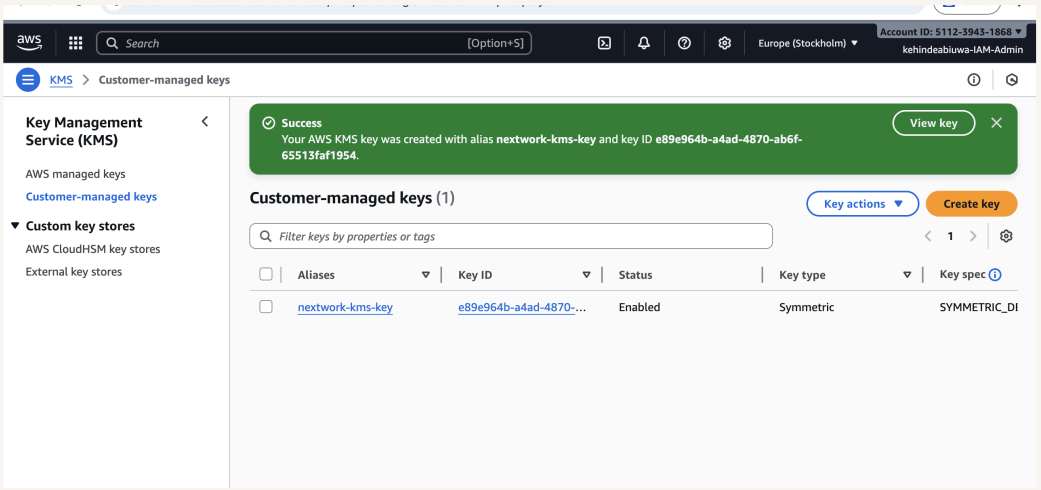
I chose to do this project today because... Something that would make learning with NextWork even better is...

Encryption and KMS

Encryption is a process that uses algorithms to convert data into a secure format called ciphertext. Only authorised users can decrypt and restore the data to its original, readable state. Companies and developers do this to to secure things like user data, transactions, files and more. Encryption keys are the secret codes computers use to lock and unlock data. Symmetric: the same key encrypts and decrypts. Asymmetric: a public key locks; a private key unlocks. In AWS KMS, your key stays inside KMS. KMS lets only approved users/apps use it, creates short-lived "data keys" to do the actual encryption (envelope encryption), can rotate keys, and logs every use for auditing.

AWS KMS is Amazon's managed service for creating, protecting, and using cryptographic keys. It lets you define who can administer a key vs who can use it, integrate encryption with services like DynamoDB/S3/EBS, rotate keys, and audit every use. Key management systems are important because they centralize key control, prevent hard-coded secrets, enable least-privilege access, automate rotation/revocation, and provide audit trails for compliance—keeping data encrypted, secure, and recoverable.

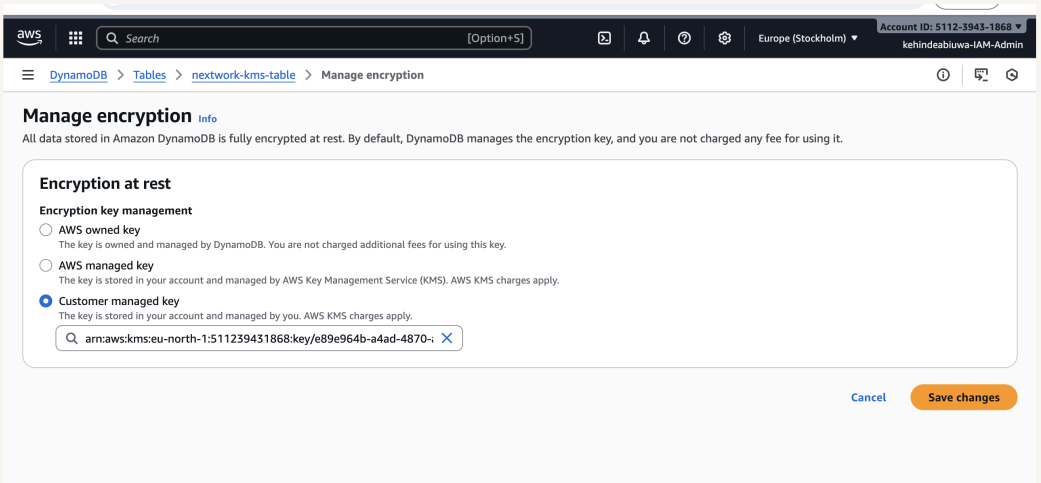
Encryption keys are broadly categorized as symmetric (one key to encrypt/decrypt) and asymmetric (public/private key pair). I set up a symmetric key because DynamoDB's SSE-KMS integration requires a KMS symmetric key, it's faster and cheaper for encrypting data at rest, and it keeps things simple


aws
Search
[Option+S]
Europe (Stockholm)
Account ID: 5112-3943-1868
kehindeabiuwa-IAM-Admin
KMS
Customer-managed keys
Key Management Service (KMS)
AWS managed keys
Customer-managed keys
Custom key stores
AWS CloudHSM key stores
External key stores
Success
Your AWS KMS key was created with alias nextwork-kms-key and key ID e89e964b-a4ad-4870-ab6f-65513faf1954.
View key
Customer-managed keys (1)
Key actions
Create key
Filter keys by properties or tags
Aliases
Key ID
Status
Key type
Key spec
nextwork-kms-key
e89e964b-a4ad-4870-...
Enabled
Symmetric
SYMMETRIC_DI

Encrypting Data

My encryption key will safeguard data in DynamoDB, which is AWS's fully managed, serverless NoSQL database

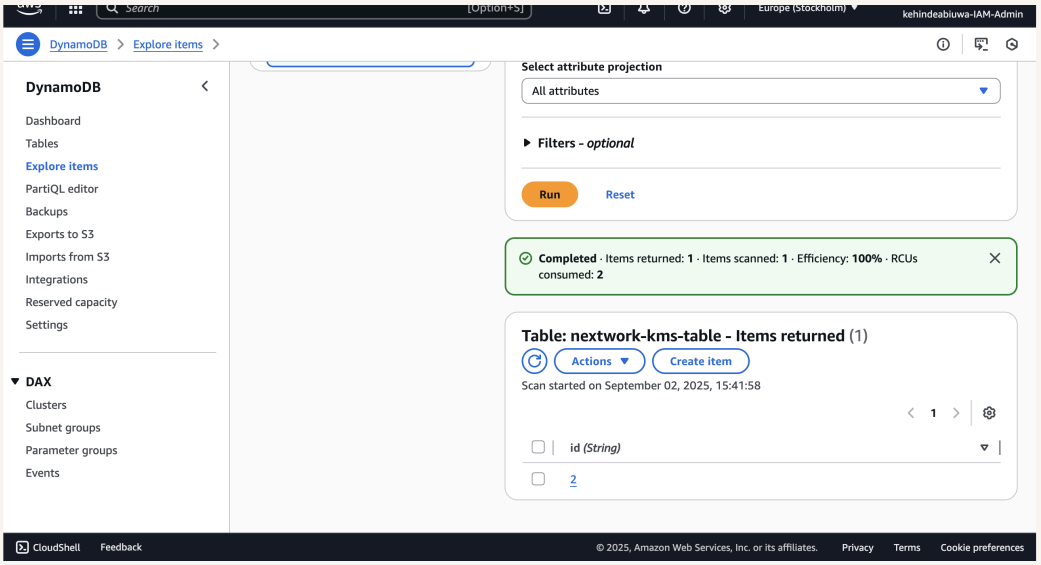
The different encryption options in DynamoDB include AWS owned key, AWS managed key (aws/dynamodb), and Customer managed key (your KMS key). Their differences are based on who controls the key, how much policy/rotation/audit control you have, and cost (AWS-owned: no extra KMS fees, least control; AWS-managed: limited control, AWS rotates yearly; customer-managed: full control/grants/rotation, standard KMS request charges). I selected Customer managed key (the key i created) so I can enforce least-privilege with key policies/grants, audit usage, and manage rotation.


aws
Search
[Option+S]
Europe (Stockholm)
Account ID: 5112-3943-1868
kehindeabiuwa-IAM-Admin
DynamoDB
Tables
nextwork-kms-table
Manage encryption
Manage encryption
Info
All data stored in Amazon DynamoDB is fully encrypted at rest. By default, DynamoDB manages the encryption key, and you are not charged any fee for using it.
Encryption at rest
Encryption key management
AWS owned key
The key is owned and managed by DynamoDB. You are not charged additional fees for using this key.
AWS managed key
The key is stored in your account and managed by AWS Key Management Service (KMS). AWS KMS charges apply.
Customer managed key
The key is stored in your account and managed by you. AWS KMS charges apply.
arn:aws:kms:eu-north-1:511239431868:key/e89e964b-a4ad-4870-
Cancel
Save changes

Data Visibility

Rather than controlling who has access to the key, KMS manages user permissions by authorizing specific actions on the key at request time—e.g., Encrypt, Decrypt, GenerateDataKey—based on the key policy (root of trust), the caller's IAM policies, and any grants/conditions (like kms:ViaService or encryption context). If the caller isn't allowed for that key/action, the request fails—even if they have database access.

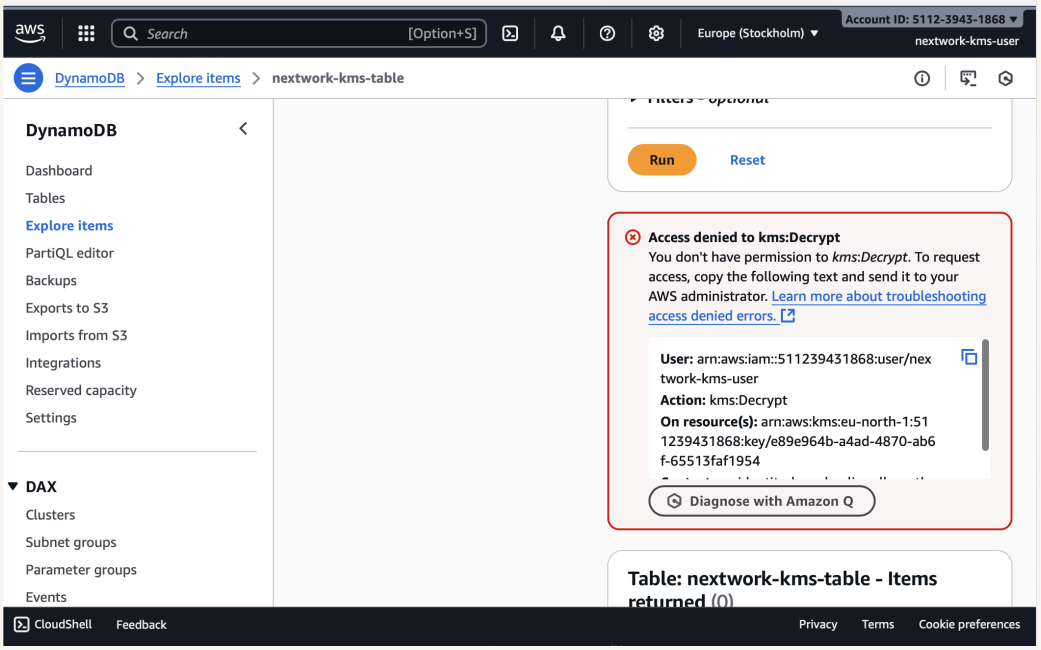
Despite encrypting my DynamoDB table, I could still see the table's items because I have permissions to use the encryption key in KMS. DynamoDB uses transparent data encryption, which means DynamoDB encrypts every item before writing it to disk and automatically decrypts it on reads server-side using your KMS key. If your IAM/KMS permissions allow Decrypt, the console and APIs return plaintext as usual (no app changes). If you don't have KMS access, the read/write fails with an access error instead of showing ciphertext.


Search
[Option+S]
Europe (Stockholm)
kehindeabiuwa-IAM-Admin
DynamoDB
Explore items
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Parameter groups
Events
Select attribute projection
All attributes
Filters - optional
Run
Reset
Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 2
Table: nextwork-kms-table - Items returned (1)
Actions
Create item
Scan started on September 02, 2025, 15:41:58
1
id (String)
2
CloudShell
Feedback
© 2025, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

Denying Access

I configured a new IAM user to test access to my encrypted DynamoDB table without KMS rights—so I can prove that encryption blocks unauthorized reads/writes. The permission policies I granted this user are AmazonDynamoDBFullAccess, but not any KMS permissions (no access to my CMK—no kms:Decrypt, kms:Encrypt, kms:GenerateDataKey, or key grants/policy membership).

After accessing the DynamoDB table as the test user, I encountered an "Access denied to kms:Decrypt" error when trying to view items, because the user has DynamoDBFullAccess but no permissions on my customer-managed KMS key, and DynamoDB needs KMS Decrypt to return plaintext. This confirmed that SSE-KMS is enforced and my least-privilege key policy is working—no KMS access, no data access.


aws
Search
[Option+S]
Europe (Stockholm)
Account ID: 5112-3943-1868
nextwork-kms-user
DynamoDB
Explore items
nextwork-kms-table
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Parameter groups
Events
Run
Reset
Access denied to kms:Decrypt
You don't have permission to kms:Decrypt. To request access, copy the following text and send it to your AWS administrator. Learn more about troubleshooting access denied errors.
User: arn:aws:iam::511239431868:user/nextwork-kms-user
Action: kms:Decrypt
On resource(s): arn:aws:kms:eu-north-1:511239431868:key/e89e964b-a4ad-4870-ab6f-65513faf1954
Diagnose with Amazon Q
Table: nextwork-kms-table - Items returned (0)
CloudShell
Feedback
Privacy
Terms
Cookie preferences

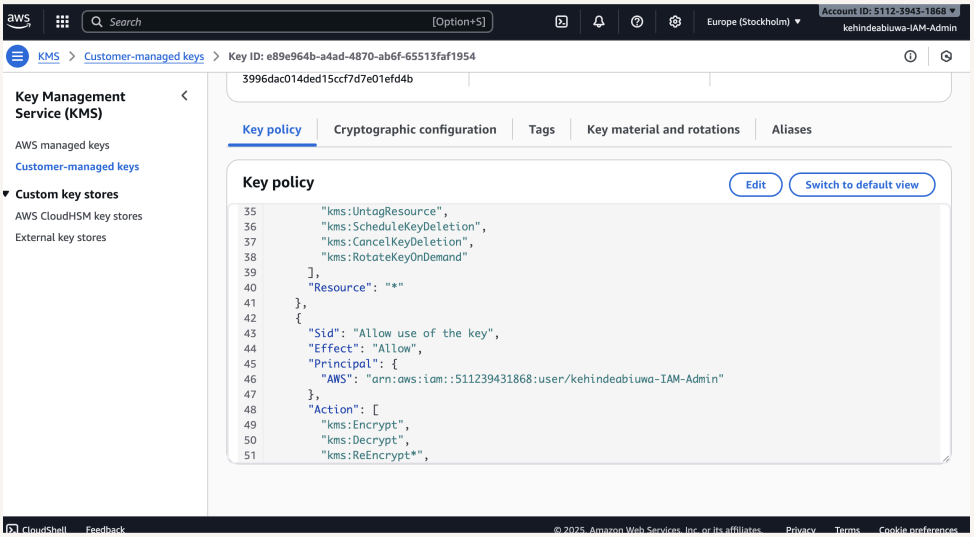
EXTRA: Granting Access

To let my test user use the encryption key, I went into the KMS console as an admin and updated the key's permissions—adding the user as a Key user with kms:Decrypt, kms:Encrypt, and kms:GenerateDataKey (I initially put them in Key administrators, but that alone doesn't allow decrypt). My key's policy was updated to include those actions for that user/principal so DynamoDB can call KMS on their behalf.

Using the test user, I retried reading items from the encrypted DynamoDB table (in the console and with a quick get-item). I observed the items returned successfully with no AccessDenied error and saw corresponding kms:Decrypt events for that user in CloudTrail/KMS. which confirmed that the key policy update worked and the test user can use the CMK to decrypt data via DynamoDB.

Encryption secures data instead of just controlling who can call the API—it protects the bytes at rest/in transit so even if storage, snapshots, or backups are accessed, they're unreadable without the key. I could combine encryption with IAM/resource policies, VPC security groups, least-privilege KMS grants, and monitoring (CloudTrail/KMS logs) to restrict who can request data, from where, and when—plus use conditions (e.g., kms:ViaService, encryption context) or client-side encryption for highly sensitive fields—to enforce defense-in-depth.

Kehinde Abiuwa
NextWork Student



The place to learn & showcase your skills

Check out nextwork.org for more projects